



Lecture
Notes

2024

Computer Graphics

Zhihao Li

Computer Science and Technology, Xidian University

zhihaoli@stu.xidian.edu.cn

CONTENTS

CHAPTER 1	BACKGROUND	PAGE 4
1.1	Development History	4
1.2	Research Topics	5
1.3	Basic Concepts	6
1.3.1	Graph VS Image	7
1.3.2	System of Computer Graphics	7
1.4	Ray-Tracing as an Example	8
1.4.1	Essence of the ray-tracing algorithm	8
CHAPTER 2	REVIEW OF LINEAR ALGEBRA	PAGE 11
2.1	Vector Multiplication	11
2.1.1	Dot(Scalar) Product	11
2.1.2	Cross(Vector) Product	12
2.1.3	Orthonormal Coordinate Frames	13
CHAPTER 3	TRANSFORMATION	PAGE 14
3.1	2D Transformations	14
3.1.1	Scale	14
3.1.2	Reflection	14
3.1.3	Shear	14
3.1.4	Rotate	15
3.1.5	Translation	15
3.1.6	Homogenous Coordinates	15
3.1.7	Affine Transformations	16
3.2	Composing and Decomposing Tranforms	17
3.2.1	Composing Transforms	17
3.2.2	Decomposing Transforms	17
3.3	3D Transformations	17
3.3.1	Homegeneous Coordinates	17
3.3.2	3D Rotations	18
3.3.3	Rodrigues's Rotation Formula	18
3.4	Pinhole Camera Model	19
3.5	Viewing Transformation	20
3.5.1	View/Camera Transformation	20
3.5.2	Projection Transformation	22

CHAPTER 4	RASTERIZATION	PAGE 26
4.1	Pixel and Screen	26
4.1.1	Screen	26
4.1.2	Screen Space	26
4.1.3	Viewport Transformation	27
4.2	Triangles Rasterization	27
4.2.1	Fundamental Shape Primitives	27
4.2.2	Sampling	27
4.2.3	Bounding Box Rasterization	28
4.3	Antialiasing and Z-Buffering	29
4.3.1	Antialiasing	29
4.3.2	Z-Buffering	32
4.4	Shading	32
4.4.1	Interaction between Lighting and Objects	33
4.4.2	Blinn-Phong Reflectance Model	34

Chapter 1

Background

1.1 Development History

- 上世纪 50 ~ 60 年代 (1950-1969)，阴极射线管 (CRT) 的出现促使了计算机图像学的兴起。阴极射线管，是一种用于显示系统的物理仪器，它是利用阴极电子枪发射电子，在阳极高压的作用下，射向荧光屏，使荧光粉发光，同时电子束在偏转磁场的作用下，作上下左右的移动来达到扫描的目的。

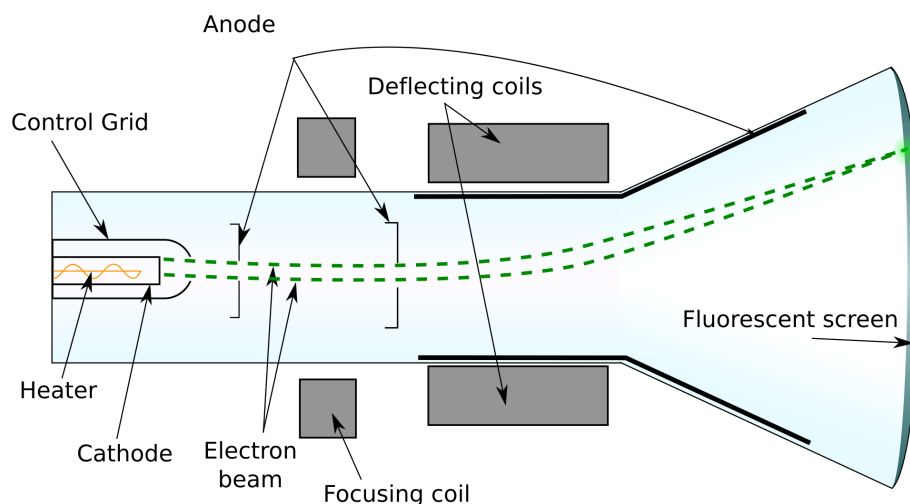
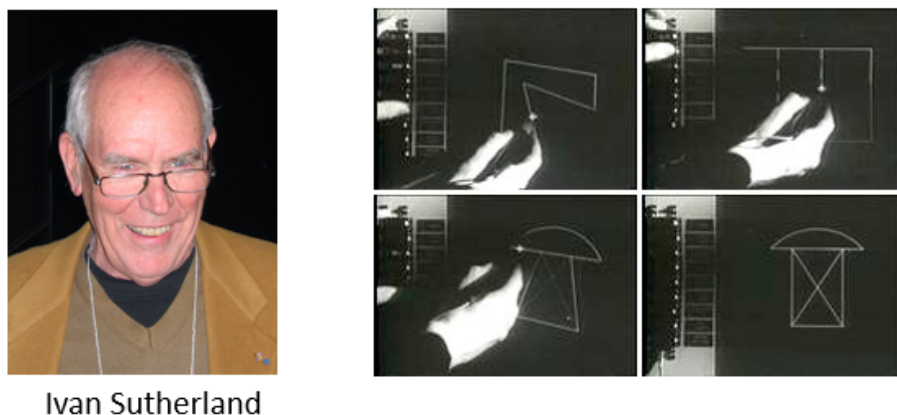


Figure 1.1: 阴极射线管中，电子束在磁场的作用下偏转

- 1950 年，第一个绘制出来的图形是由美国测绘员 Ben Laposky 通过操控模拟电子束在显示屏上呈现，这是最早的图形绘制设备，第一个通过阴极管绘制在计算机中的图形。
- 1959 年，美国通用和 IBM 公司发明了第一台工业 CAD 系统，帮助工程师进行车辆的设计，从此 CAD 逐渐发展起来。
- 1960 年，计算机图形学 (Computer Graphics) 是波音公司的工程师 Fetter 最早提出的。该概念主要用来描述飞机驾驶室模拟设计。

- 1963 年，MIT 博士生的 Ivan Sutherland（后获图灵奖，Coons 奖（图形学领域终身成就奖））设计了 Sketchpad，被公认为计算机图形学的起源。Sketchpad，赋予计算机图形处理的能力，使计算机拥有人类右脑的功能。



Ivan Sutherland

Figure 1.2: Ivan Sutherland 以及设计的 Sketchpad



Figure 1.3: 人类左脑符号处理以及右脑图像处理的能力

1.2 Research Topics

1. 自然的三维交互技术：实现更灵活的三维交互输入，从传统鼠标、单点触摸、多点触摸的二维接触式交互方式转变为基于手势、姿态等的三维非接触式交互方式。
2. 直观的三维编辑技术：实现艺术家模式的三维模型制作与编辑，通过画笔手绘等常见且直观的艺术创作手法，对三维模型进行建模或修改。
3. 富有感情的机器人技术：实现具有人类相似情感表现的机器人，用人工的方法和技术赋予计算机或机器人以人类式的情感，使之在工作时具有表达、识别和理解喜怒哀乐，模仿、延伸和扩展人的情感的能力。
4. 绘画辅助技术：学习和控制绘制过程以及理解绘制物的语义，通过直线、曲线、笔画、素描等不同形式的绘画工具，让计算机更加智能地辅助进行人工参与的绘画创作。

5. 数学辅助技术：实现更便捷的数学符号和公式的输入、输出，通过图形方式辅助论文编辑、数学软件、数学手写板、数学推理等。
6. 海量数据的可视化技术：海量数据的视觉感知，借助面向海量数据的可视化技术，从根本上改变人们表示、分析和理解海量复杂数据的方式。
7. 虚拟导游技术：将虚拟角色融入吃、穿、住、用、行的虚拟现实场景，利用计算机提供的智能导游服务取代人力服务，将能大大提高服务质量。
8. 高沉浸感技术：真实感的虚拟环境沉浸式体验和交互，以虚拟洞穴、虚拟办公室、VR 眼镜等为代表的虚拟现实技术，需要用户专注在当前的目标情境下感到愉悦和满足，而忘记真实世界的情境。
9. 增强现实技术：叠加虚拟物体到真实世界，在屏幕上把虚拟世界套在现实世界并进行互动，使用户就不必在其他设备上查看相关信息，以便腾出双手进行其他任务。
10. 混合现实技术：形成与真实世界无缝融合的高品质的完全沉浸式虚实融合的环境，通过交互构建现实世界、虚拟世界和用户之间的信息回路。

1.3 Basic Concepts

计算机图像学研究真实或者虚拟世界的图形表示以及人机交互，是涉及计算机、数学、物理等学科的交叉领域。

Claim 1.3.1 Core Content

- 建模：在计算机中通过几何、图像、视频等形式构建和表示物体模型；
- 绘制：通过可视媒介（图像、视频、全息等）呈现计算机中存储的物理模型；
- 交互：通过计算机输入、输出设备，以有效的方式对计算机中的模型进行操作；

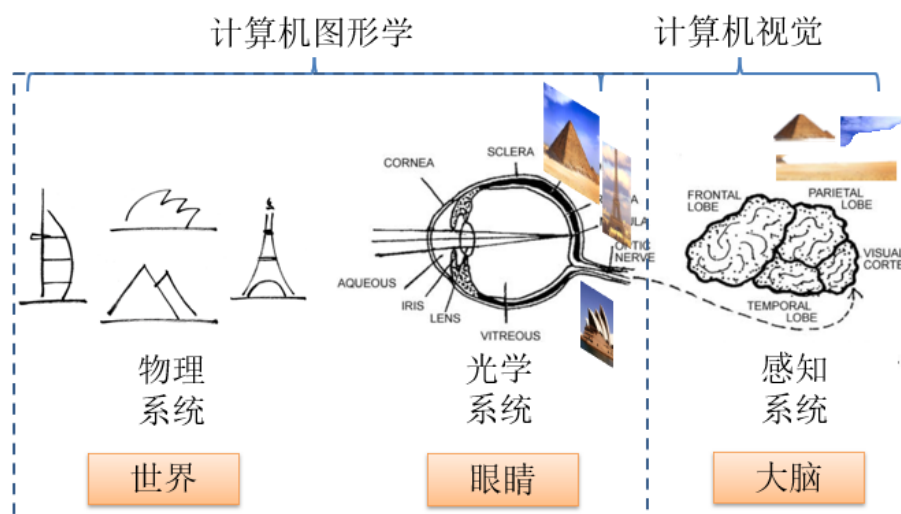


Figure 1.4: 计算机图像学与计算机视觉

1.3.1 Graph VS Image

Definition 1.3.1: 图形

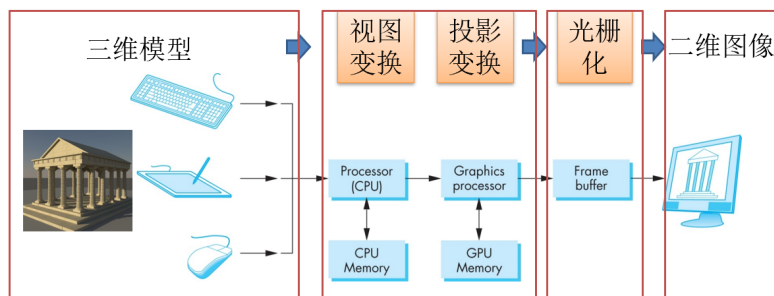
由点、线、面等基本几何元素作为“图元”构成，通过建模、测量等方式获取。

Definition 1.3.2: 图像

由像素构成，通过照相、扫描等方式获取。

Definition 1.3.3: 光栅化

二维和三维图形需要通过光栅化转化为图像进行屏幕显示，光栅化是图形学中一个重要的概念，它是将几何图形转化为像素点阵的过程。



计算机图形学系统

Figure 1.5: 图形流水线

1.3.2 System of Computer Graphics

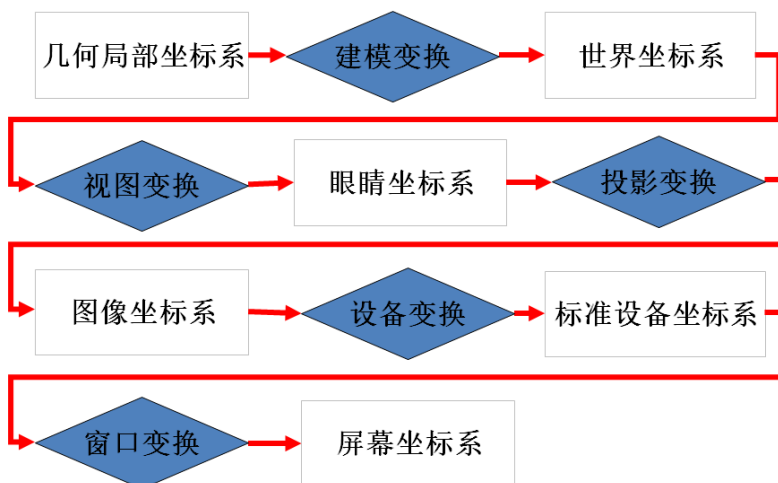


Figure 1.6: 计算机图形学系统

1.4 Ray-Tracing as an Example

计算机图形学——a domain fueled by passion, 创造有意义的计算机图像是一件非同寻常的事情。

- Image plane: 相当于艺术家的画布 canvas, 用作投影媒介。
- 各向异性反射 (anisotropic reflection): 某些材料表现出有组织的宏观结构, 这些宏观结构在特定方向上引导光反射的现象。
- 光线追踪分为前向追踪 (Forward ray tracing or light tracing) 以及反向追踪 (Backward ray-tracing or eye tracing)。
- 前向追踪: 从光源投射光线, 一直持续到所有像素都经过适当调整, 最终形成计算机生成的图像, 即追踪光子从光源到观察者的路径。但是这种方法的难以实际应用, 因为光子在物体表面发射后只有很小的概率才能进到人的眼睛中, 这意味着需要发射大量的光子才有可能获得整幅场景的图像。自然界中的光子都是在以光速运动, 而且数量难以估计, 这种特性在计算机中需要极大的计算代价才能够模仿, 因此对于复杂或庞大的场景束手无策。
- 反向追踪: 逆着光的传播方向, 从接收者发投射光线到物体, 一旦接触到物体, 我们通过接触点向光源发出另一条光线 (阴影光线) 来评估它接收到的光线。如果阴影光线接触到了另外一个物体, 说明最初的接触点是阴影的, 没有接收到光线。从眼睛 (相机) 发射到场景的吐舌光纤在计算机图像学中又被称为主光线 (Primary ray)、可见光线 (Visibility ray) 或相机光线 (Camera ray)。

1.4.1 Essence of the ray-tracing algorithm

光线追踪的本质是逐像素地 (pixel by pixel) 渲染一张图像, 对于每一个像素, 发射一条 primary ray 到场景中, 它的方向是通过眼睛到像素中心的直线, 并选择 primary ray 与场景中最近物体的交点处发射另外一条 shadow ray 朝向光源。

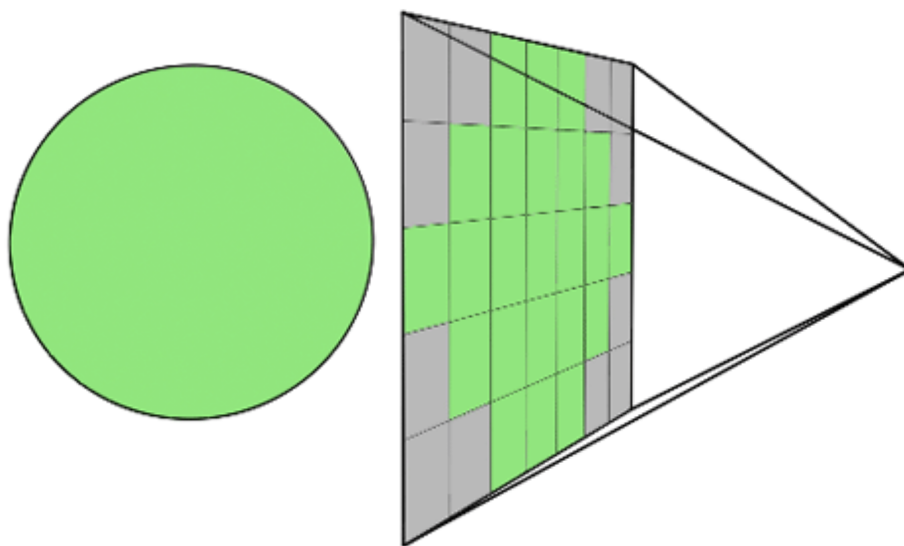


Figure 1.7: Rendering a frame involves dispatching a primary ray for every pixel within the frame buffer.

光线追踪算法，计算非常耗费时间，相比于 Z-Buffer 算法是较为缓慢的，整体上可以分为可见性确定和着色两部分。一个简单的算法伪代码如下，

```

for (int j = 0; j < imageHeight; ++j) {
    for (int i = 0; i < imageWidth; ++i) {
        // Determine the direction of the primary ray
        Ray primRay;
        computePrimRay(i, j, &primRay);
        // Initiate a search for intersections within the scene
        Point pHit;
        Normal nHit;
        float minDist = INFINITY;
        Object *object = NULL;
        for (int k = 0; k < objects.size(); ++k) {
            if (Intersect(objects[k], primRay, &pHit, &nHit)) {
                float distance = Distance(eyePosition, pHit);
                if (distance < minDist) {
                    object = &objects[k];
                    minDist = distance; // Update the minimum distance
                }
            }
        }
        if (object != NULL) {
            // Illuminate the intersection point
            Ray shadowRay;
            shadowRay.direction = lightPosition - pHit;
            bool isInShadow = false;
            for (int k = 0; k < objects.size(); ++k) {
                if (Intersect(objects[k], shadowRay)) {
                    isInShadow = true;
                    break;
                }
            }
            if (!isInShadow)
                pixels[i][j] = object->color * light.brightness;
            else
                pixels[i][j] = 0;
        }
    }
}

```

光线追踪算法除了计算简单，还能够无缝模拟复杂的光学效应，例如反射和折射。当反射和折射同时发生时，就需要混合这两种效应，其中重要的是混合率（blend ratio）。

混合率不仅受到入射角的影响，还受到表面法线和材料折射率等因素的影响。Fresnel equation 提供了一个确定反射和折射适当组合的公式，需要材料的折射率以及交点处入射光线与法线的夹角，并输出反射和折射的混合值。

```
// compute reflection color
color reflectionColor = computeReflectionColor();

// compute refraction color
color refractionColor = computeRefractionColor();

float Kr; // reflection mix value
float Kt; // refraction mix value

// Calculate the mixing values using the Fresnel equation
fresnel(refractiveIndex, normalHit, primaryRayDirection, &Kr, &Kt);

// Mix the reflection and refraction colors based on the Fresnel
// equation. Note Kt = 1 - Kr
glassBallColorAtHit = Kr * reflectionColor + Kt * refractionColor;
```

Chapter 2

Review of Linear Algebra

2.1 Vector Multiplication

2.1.1 Dot(Scalar) Product

点积又称为标量积 (Scalar Product) / 数量积，是两个向量对应元素乘积之和，通常被称为欧几里得空间的内积（或投影积）。

- 代数定义

对于两个向量 $\vec{a} = [a_1, a_2, \dots, a_n]$, $\vec{b} = [b_1, b_2, \dots, b_n]$ 的点积定义为，

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n \quad (2.1)$$

如果是 $\vec{a}, \vec{b}$ 都是列向量，可以进一步写为向量乘积的形式：

$$\vec{a} \cdot \vec{b} = \vec{a}^T \vec{b} \quad (2.2)$$

- 几何定义

在欧几里得空间中，向量被定义为具有大小和方向的几何对象，其点积定义为：

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos \theta \quad (2.3)$$

- 运算性质

$$\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a} \quad (2.4)$$

$$\vec{a} \cdot (\vec{b} + \vec{c}) = \vec{a} \cdot \vec{b} + \vec{a} \cdot \vec{c} \quad (2.5)$$

$$(k\vec{a}) \cdot \vec{b} = \vec{a} \cdot (k\vec{b}) = k(\vec{a} \cdot \vec{b}) \quad (2.6)$$

- 常见用法

计算向量长度：向量的长度等于向量本身的点积的平方根，即 $\|\vec{a}\| = \sqrt{\vec{a} \cdot \vec{a}}$

计算向量夹角：两个向量夹角的余弦等于它们的点积与它们长度的乘积的商，即 $\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$

判定向量正交：如果两个向量正交，则点积 $\vec{a} \cdot \vec{b} = 0$

计算向量投影： $\vec{b}$ 在 $\vec{a}$ 上的投影， $\vec{b}_{\perp} = \frac{\vec{a} \cdot \vec{b}}{\vec{a} \cdot \vec{a}} \vec{a} = \frac{\|\vec{b}\| \cos \theta}{\|\vec{a}\|} \vec{a}$

测量向量 close 程度： $\vec{a} \cdot \vec{b} > 0$ 表示两个向量同向， $\vec{a} \cdot \vec{b} < 0$ 表示两个向量反向，数值表示同向或反向的程度。

2.1.2 Cross(Vector) Product

叉积又称为向量积，其结果是一个向量，与两个初始的向量相互正交，方向可以由右手定则/右手螺旋法则确定。

- 基本定义

叉积 $\vec{a} \times \vec{b}$ 定义为与 $\vec{a}$ 和 $\vec{b}$ 都垂直（正交）的向量 $\vec{c}$ ，即

$$\vec{a} \times \vec{b} = \|\vec{a}\| \|\vec{b}\| \sin \theta \vec{n} \quad (2.7)$$

其中， $\vec{n}$ 是垂直于包含 $\vec{a}$ 和 $\vec{b}$ 的平面的单位向量。

- 运算性质

$$\vec{x} \times \vec{y} = -\vec{y} \times \vec{x} \quad (2.8)$$

$$\vec{a} \times \vec{a} = 0 \quad (2.9)$$

$$\vec{a} \times (\vec{b} + \vec{c}) = \vec{a} \times \vec{b} + \vec{a} \times \vec{c} \quad (2.10)$$

$$\vec{a} \times (k\vec{b}) = k(\vec{a} \times \vec{b}) \quad (2.11)$$

- 图形学中用法

确定左与右，以及确定内和外。如下图所示，根据右手螺旋定则可以判定，由 $\vec{a} \times \vec{b}$ 方向与 z 轴方向一致，因此可以判断 $\vec{b}$ 在 $\vec{a}$ 的左边，同理由 $\vec{b} \times \vec{a}$ 方向与 z 轴方向相反，因此可以判断 $\vec{a}$ 在 $\vec{b}$ 的右边。其中主要的判断依据是，右手螺旋定则是从右手边向左手边旋转，以此确定 z 轴的方向，如果有一个向量与 z 轴方向一致，那么它一定遵循右手螺旋定则。

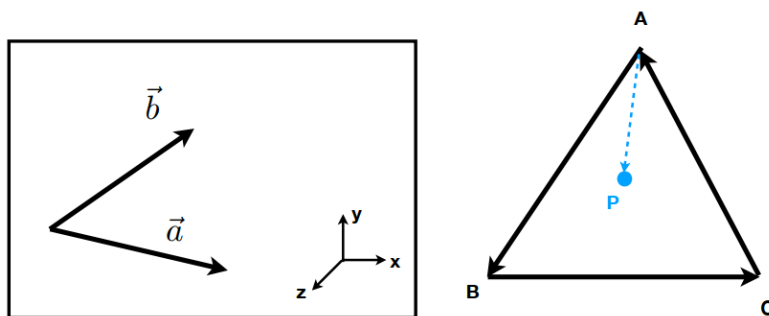


Figure 2.1: 叉积在图形学中的应用

对于三角形中，按照向量 $\vec{AB} \times \vec{AP}$ 确定 P 点在 $\vec{AB}$ 的左侧还是右侧，同理依次确定 P 点相对于 $\vec{BC}$ 以及 $\vec{AC}$ 的位置，如果 P 点都在三条边的左侧或右侧，则可以判断 P 点是在三角形的内部，否则在三角形的外部。

2.1.3 Orthonormal Coordinate Frames

正交坐标系，任何一组包含 3 个三维向量的向量组，如果满足如下条件，即可作为正交坐标系的标准正交基，

$$\|\vec{u}\| = \|\vec{v}\| = \|\vec{w}\| = 1 \quad (2.12)$$

$$\vec{u} \cdot \vec{v} = \vec{v} \cdot \vec{w} = \vec{u} \cdot \vec{w} = 0 \quad (2.13)$$

$$\vec{w} = \vec{u} \times \vec{v} (\text{right-handed}) \quad (2.14)$$

那么，对于向量空间 (R^3) 中的任意向量 $\vec{p}$ 都可以进行正交分解为，

$$\vec{p} = (\vec{p} \cdot \vec{u})\vec{u} + (\vec{p} \cdot \vec{v})\vec{v} + (\vec{p} \cdot \vec{w})\vec{w} \quad (2.15)$$

Chapter 3

Transformation

对于变换在图形学中主要应用在建模中相机的变换、模型的旋转、模型缩放以及 3D 到 2D 的投影变换。

3.1 2D Transformations

对于二维场景中的物体变换，在初高中教科书中都有涉及这种二维直角坐标系下的物体变换，包括平移、旋转、对称、缩放等操作，尤其是高中学过的仿射变换，在图形学中也会有用到。相比于高中所学，图形学中的 2D 变换更加侧重于利用矩阵表示图形变换。

3.1.1 Scale

缩放是指对于一个图形的所有维度进行伸缩，主要表现为延展和紧缩，具体需要关注于缩放矩阵，

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (3.1)$$

其中， s_x 与 s_y 分别表示对 x 和 y 维度施加相同或不同的缩放系数。

3.1.2 Reflection

对称是指将图形沿着某一维度进行镜像对称，例如如果按 y 轴做镜像对称，其对应的对称矩阵为，

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (3.2)$$

3.1.3 Shear

错切是一种比较新颖的图形变换方式，更加直观地讲可以认为这种变换方式是一种图形的倾斜，例如对于下图中的图形，主要是沿 x 轴进行倾斜， y 坐标保持不变，其对应的错切矩阵为，

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & a \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (3.3)$$

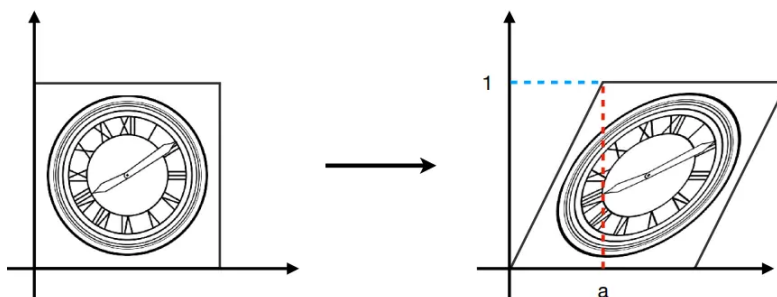


Figure 3.1: 错切图形变换

3.1.4 Rotate

旋转就是将图形绕某个中心以顺时针或逆时针方向进行旋转，但是在图形学中，默认的旋转方式都是**图形绕坐标系原点以逆时针方向进行旋转**，这样的旋转方式是最简单直观的，而其他更加复杂的旋转方式都是通过图形变换分解操作来实现，因此掌握默认的旋转方式即可。

默认的图形旋转方式如下图所示，其对应的旋转矩阵为，

$$R_{\theta} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (3.4)$$

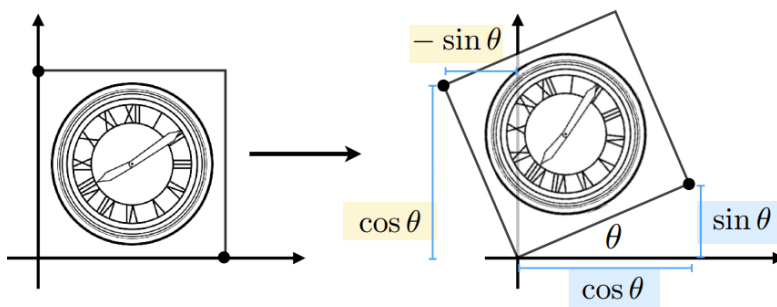


Figure 3.2: 默认的图形旋转方式

3.1.5 Translation

平移操作是指将图形按某个方向平行移动一段距离，实际上是对图形的坐标都增添一个位移量，具体表述为

$$x' = x + t_x \quad (3.5)$$

$$y' = y + t_y \quad (3.6)$$

3.1.6 Homogenous Coordinates

我们希望利用矩阵的性质找到这些图形变换的统一表示形式，即 $x' = Mx$ 。但是，平移操作是对原始坐标进行增量操作，无法表示成矩阵相乘的形式，即

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} \quad (3.7)$$

解决方法就是齐次坐标 (Homogenous Coordinates)，在计算机图形学和几何学中，齐次坐标常用于表示平移、旋转和缩放等几何变换，使得这些变换可以通过矩阵运算来统一处理。齐次坐标引入了一个额外的维度，以便在同一框架内表示线性和仿射变换。

齐次坐标增添了额外的 w 维度，分别表示坐标点和向量的形式，

- 2D Point = $(x, y, 1)^T$

- 2D Vector = $(x, y, 0)^T$

当用齐次坐标重新表示平移操作时，可以得到统一的表示形式

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} \quad (3.8)$$

齐次坐标的有效性解释: 对于 Vector 而言，其 w 维度为 0 可以保持向量线性运算的性质，而对于 Point 而言，其 w 维度为 1 可以构造出向量以及获得坐标点，具体表示如下：

- vector + vector = vector
- point - point = vector
- point + vector = point
- point + point = middle-point

在齐次坐标中，对于每个坐标点更加广义的定义为，

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} = \begin{bmatrix} x/w \\ y/w \\ 1 \end{bmatrix}, w \neq 0 \quad (3.9)$$

3.1.7 Affine Transformations

仿射变换主要包括线性变换以及平移，即 affine map = linear map + translation，公式表示为：

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix} \quad (3.10)$$

如果采用齐次坐标表示，仿射变换可以统一为：

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (3.11)$$

具体而言重新表示上述各种图形变换方式对应的变换矩阵：

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}, R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}, T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad (3.12)$$

3.2 Composing and Decomposing Transforms

合成和分解图形变换方式能够让我们形成更加复杂的图形变换方式，以及对于复杂变换分解为基本变换的叠加。

3.2.1 Composing Transforms

假设有一组仿射变换序列， $A_1, A_2, A_3, \dots$ ，可以通过矩阵相乘的形式将这些仿射变换合成更加一个复杂的变换，然后再对坐标点进行变换，这有助于提高处理性能，

$$A_n(\dots A_2(A_1(x))) = A_n \cdots A_2 \cdot A_1 \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (3.13)$$

其中由于矩阵乘法的性质，图形变换的顺序不可调换，可以通过结合预先处理出变换序列的变换矩阵，这样简洁的变换形式的好处主要源自采用齐次坐标统一了各种图形变换方式。

$$A_n(\dots A_2(A_1(x))) = (A_n \cdots A_2 \cdot A_1) \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = A \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (3.14)$$

3.2.2 Decomposing Transforms

特殊的图形旋转方式：当图形不再围绕坐标系中心按照逆时针旋转时，我们可以将这种复杂的旋转方式分解为基本的旋转方式，对应的矩阵表示形式为： $T(c) \cdot R(\alpha) \cdot T(-c)$

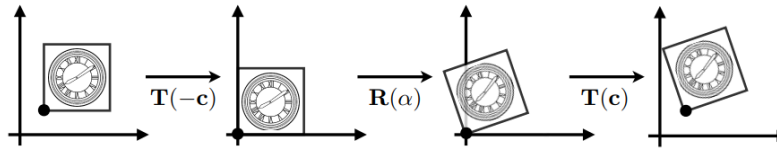


Figure 3.3: 复杂图形旋转方式的分解

3.3 3D Transformations

3.3.1 Homogeneous Coordinates

在建立了 2D Transforms 的齐次坐标变换后，可以直接得到对应于 3D Transforms 的齐次坐标表示，

- 3D point = $(x, y, z, 1)^T$
- 3D vector = $(x, y, z, 0)^T$

广义的齐次坐标表示：对于点 (x, y, z, w) ($w \neq 0$) 而言，齐次坐标的含义表示为 $(x/w, y/w, z/w)$ 。

3D 仿射变换的齐次坐标表示

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c & t_x \\ d & e & f & t_y \\ g & h & i & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad (3.15)$$

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, T(t_x, t_y, t_z) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.16)$$

对于三维旋转，主要考虑绕 x, y, z 轴的基本旋转变换，如下式所示，对于围绕的坐标轴坐标保持不变，其他的维度所构成的旋转变换子矩阵就是 2D 的绕轴旋转变换矩阵。

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R_y(\alpha) = \begin{bmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R_z(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.17)$$

其中，对于绕 y 轴旋转的矩阵变换 $R_y(\alpha)$ 由于 $z \times x$ 得到 y 轴，因此对于变换 $\sin \alpha$ 额外取负。

3.3.2 3D Rotations

对于任意的 3D 旋转变换，可以将其分解为绕 x, y, z 轴的基本变换的组合，即

$$R_{xyz}(\alpha, \beta, \gamma) = R_x(\alpha)R_y(\beta)R_z(\gamma) \quad (3.18)$$

这也被称为欧拉角 (Euler Angles)，三个旋转动作分别称为：roll, pitch, yaw，对应的旋转轴相互垂直，就是取得 x, y, z 轴，如下图所示：

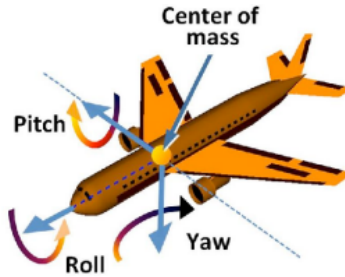


Figure 3.4: 欧拉角对应的三个旋转轴

3.3.3 Rodrigues's Rotation Formula

对于绕特定轴 $\vec{n}$ 旋转任意角度 α 计算对应的旋转变换矩阵，可以通过分解的思想：先将旋转轴平移到坐标轴上，然后绕坐标轴旋转，最后将旋转后的图形全部移到原来的旋转轴上。该公式形式化计算为，

$$R(\vec{n}, \alpha) = \cos(\alpha)I + (1 - \cos(\alpha))\vec{n}\vec{n}^T + \sin(\alpha) \begin{bmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{bmatrix} \quad (3.19)$$

3.4 Pinhole Camera Model

在了解各个坐标系的转换前，首先需要理解图形学中最常用的相机模型，其原理和小孔成像是类似的，如图 3.5 所示。

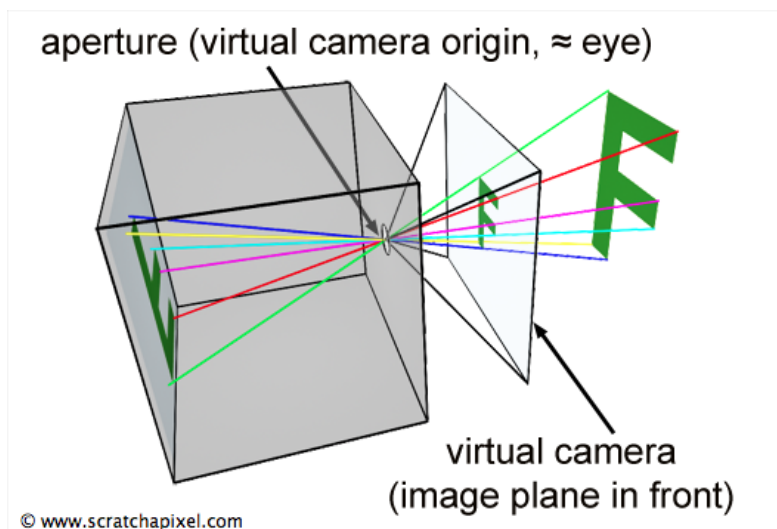


Figure 3.5: Principle of a pinhole camera.

图像学中的相机，其成像平面是在小孔（即相机光圈）前方，而小孔成像是在小孔后方的成像平面上投影出倒立的图像。正立的投影图像是希望获取的，便于后续处理，而小孔前成像就刚好满足这一点。

当我们深入考虑到成像平面大小以及成像范围时，就需要一个专门的模型来进行定义，事实上伴随着一些概念定义，还是非常形式化的。

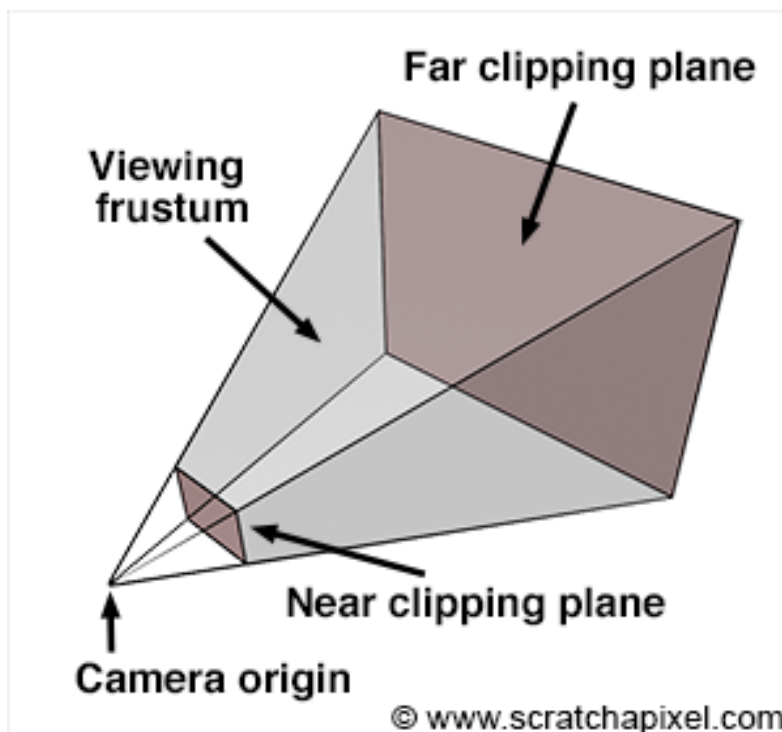


Figure 3.6: Any object within the viewing frustum is visible.

如图 3.6 所示的形似棱锥的模型，就是相机的可视范围，需要了解一些相关的概念。

1. Near Clipping Plane: 近平面，相机前方的平面，也是成像平面。
2. Far Clipping Plane: 远平面，相机前方更远处的平面。在真实的相机中，一般指无限远的平面，但为了简化计算，在图像学中将其设为有限的距离。
3. Viewing Frustum: 视锥体，近平面与远平面之间的空间，也即是相机的可视范围，只有在这个范围内的物体才能被投影到相机的成像平面上。

3.5 Viewing Transformation

观测变换主要包括 View(视图)/Camera Transformation 以及 Project(投影) Transformation，其中投影变换又包括 Orthographic(正交) Projection 以及 Perspective(透视) Projection.

3.5.1 View/Camera Transformation

类比相机照相过程，Model Transformation 是寻找位置、安排人员的过程，而 View Transformation 是寻找角度、摆放相机的过程，Projection Transformation 是最后从 3D 场景中拍摄照片，即投影到 2D 平面上的过程。

相机属性定义

在视图变换中，主要是变换相加的过程，那么需要首先定义相机在三维空间中的属性：

- Position $\vec{e}$: 相机的位置坐标;
- Look-at Direction g : 相机的朝向决定拍摄的方向, 即 yaw 旋转;
- Up Direction t : 相机围绕拍摄方向的旋转角度, 决定拍摄的视角, 即 roll 旋转;

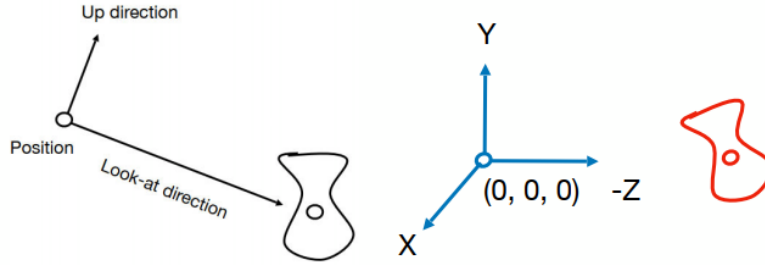


Figure 3.7: 相机的摆放方向

如图 4.1 的右图所示, 相机和物体同时移动时, 拍摄的内容将不保持不变, 因此可以考虑将相机固定在一个特殊的位置, 保持物体同时进行移动即可完成视图变换。

固定的相机位置: 固定在原点, 且保持 Up Direction 与 y 轴保持一致, Look-at Direction 与 $-z$ 轴保持一致。

视图变换矩阵计算

在固定相机位置的过程中, 其实就是对视图进行变换的过程, 固定好相机后, 物体的位置也随即确定。视图变换矩阵 M_{view} 是将相机移至之前固定位置的一系列操作的组合, 即

- Translates to origin
- Rotates g to $-z$
- Rotates t to y
- Rotates $g \times t$ to x

由于先平移再旋转, 即可得到 $M_{view} = R_{view}T_{view}$, Translate to origin 对应的平移矩阵可以认为将相机坐标 $\vec{e}$ 平移 $-\vec{e}$ 后得到, 即

$$T_{view} = \begin{bmatrix} 1 & 0 & 0 & -x_e \\ 0 & 1 & 0 & -y_e \\ 0 & 0 & 1 & -z_e \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.20)$$

而旋转 Rotate g to $-z$, t to y 以及 $g \times t$ to x 可以通过逆变换来求解, 即 x to $g \times t$, y to t 以及 z to $-g$ 这里注意不仅两个坐标系的轴进行了调换, 对应的变的顺序也做了调换。具体的视图变换矩阵计算如下,

$$R_{view}^{-1} = \begin{bmatrix} x_{g \times t} & x_t & x_{-g} & 0 \\ y_{g \times t} & y_t & y_{-g} & 0 \\ z_{g \times t} & z_t & z_{-g} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \Rightarrow R_{view} = \begin{bmatrix} x_{g \times t} & y_{g \times t} & z_{g \times t} & 0 \\ x_t & y_t & z_t & 0 \\ x_{-g} & y_{-g} & z_{-g} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.21)$$

两点解释

1. 对于 R_{view}^{-1} 而言, 可以取 xyz 坐标系下各坐标轴的单位向量, 记为 $\vec{x} = [1, 0, 0]^T, \vec{y} = [0, 1, 0]^T, \vec{z} = [0, 0, 1]^T$, 则对于旋转变换需要满足:

$$\begin{cases} R_{view}^{-1} \cdot \vec{x} = \vec{g} \times \vec{t} \\ R_{view}^{-1} \cdot \vec{y} = \vec{t} \\ R_{view}^{-1} \cdot \vec{z} = -\vec{g} \end{cases} \quad (3.22)$$

因此, 根据上面的等式要求, 即可得到 R_{view}^{-1} .

2. 对于 R_{view} 而言, 由于旋转矩阵是正交矩阵, 其矩阵的逆等于矩阵的转置, 因此可以得到 $R_{view}^T = R_{view}^{-1}$, 进一步求出 R_{view} ,

$$R_{view} = [R_{view}^T]^T = [R_{view}^{-1}]^T \quad (3.23)$$

因此, 视图变换是变换相机以及关联的全部物体直到相机的位置固定在原点、向上方向与 y 轴保持一致、朝向与 $-z$ 轴保持一致。视图变换的目的是为了更好地做投影变换。

3.5.2 Projection Transformation

投影变换包括透视变换和正交变换, 透视变换主要是实际观测到的图形投影, 相机捕获到的光线不是平行光, 造成投影过程中图形线条发生收缩现象; 正交变换主要用于工程制图, 是理想地对物体抽象建模, 不会因为观察视角造成投影图形线条的变化。

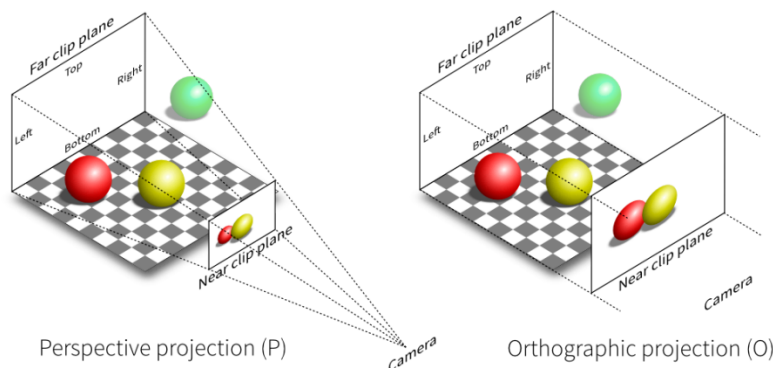


Figure 3.8: 透视投影与正交投影对比图

Orthographic Projection

正交投影/平行投影实际上将一个实际的立方体图形缩放至一个标准的立方体 $[-1, 1]^3$, 先将原立方体中心平移至坐标原点, 然后各自对 xyz 轴的范围进行缩放, 使之处于 $[-1, 1]$ 之间。

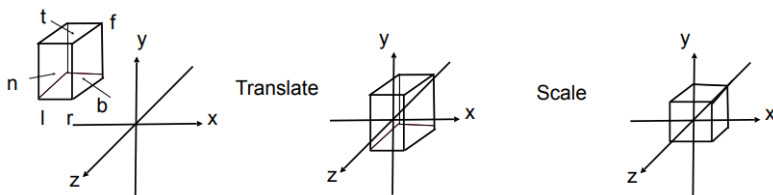


Figure 3.9: 正交投影变换

形式化的变换矩阵表示为,

$$M_{ortho} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{n-f} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -\frac{r+l}{2} \\ 0 & 1 & 0 & -\frac{t+b}{2} \\ 0 & 0 & 1 & -\frac{n+f}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.24)$$

因此, 正交变换之后会对物体形状造成拉伸, 都被变换到 $[-1, 1]^3$ 立方体之间。

Perspective Projection

透视投影满足近大远小的性质, 在图形学中非常常见。计算透视投影, 可以将其转换为计算透视投影到正交投影的变换, 而正交投影已经计算出。如下图所示, 对于圆锥街头体 (Frustum) 图形, 将远平面进行压缩直至变为一个立方体, 并保证面中心保持不变。

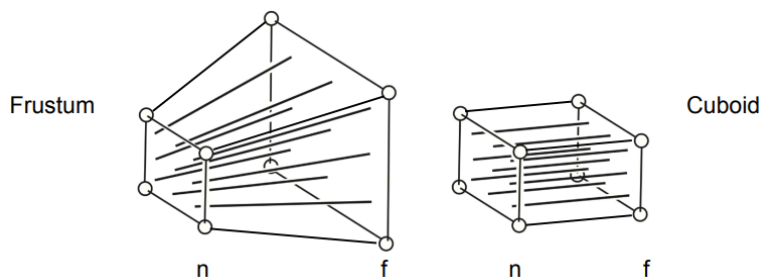


Figure 3.10: 透视投影变换为正交投影

计算对应变换前后的坐标关系, 如下图对物体投影的分析所示, 根据三角形相似性可以计算出远平面压缩后对应的 y 坐标为: $y' = \frac{n}{z}y$.

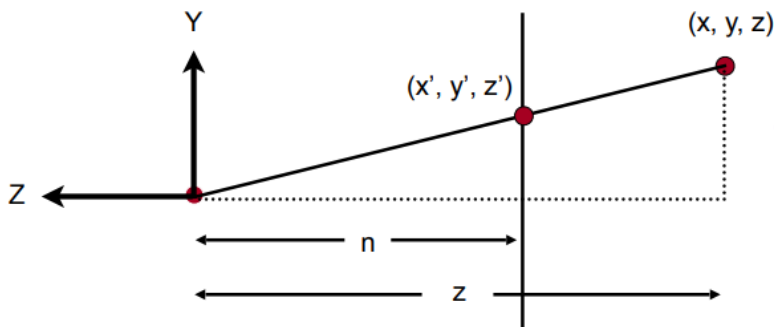


Figure 3.11: 透视投影变换为正交投影

因此可以得到, 远平面变换后的坐标为,

$$y' = \frac{n}{z}y, x' = \frac{n}{z}x \quad (3.25)$$

但是由于物体压缩会导致物体上各点的分布变化, 对应的 z 坐标就无法确切的知道。在齐次坐标系下,

有如下关系,

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} nx/z \\ ny/z \\ \text{unknown} \\ 1 \end{bmatrix} == \begin{bmatrix} nx \\ ny \\ \text{still unknown} \\ z \end{bmatrix} \quad (3.26)$$

那么我们一定知道, 透视到正交投影变换矩阵一定会满足下式,

$$M_{persp \rightarrow ortho} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} nx \\ ny \\ \text{unkonwn} \\ z \end{bmatrix} \Rightarrow M_{persp \rightarrow ortho} = \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ ? & ? & ? & ? \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (3.27)$$

利用其它信息来计算剩余的矩阵元素,

1. 近平面上的点不会改变

对于近平面上的点, 用 n 替换 z 可以得到, 坐标点 $[x, y, n, 1]^T$, 应用 $M_{persp \rightarrow ortho}$ 得到如下公式:

$$M_{persp \rightarrow ortho} \cdot \begin{bmatrix} x \\ y \\ n \\ 1 \end{bmatrix} = \begin{bmatrix} nx \\ ny \\ \text{unknown} \\ n \end{bmatrix} = \begin{bmatrix} x \\ y \\ n \\ 1 \end{bmatrix} \quad (3.28)$$

根据变换前后坐标不变的性质, 可以得出 unknown 是一定是 n^2 , 因为 $[nx, ny, n^2, n] == [x, y, z, 1]$ 。因此, 我们可以得到推导出的关系式,

$$[0, 0, A, B] \begin{bmatrix} x \\ y \\ n \\ 1 \end{bmatrix} = n^2 \Rightarrow An + B = n^2 \quad (3.29)$$

其中 n^2 与 x, y 无关, 所有向量的前两项一定为 0.

2. 远平面上的点 z 坐标不会改变

对于远平面上的点, 可以选取一个 z 轴上的交点 $[0, 0, f, 1]^T$ 进行计算,

$$\begin{bmatrix} 0 \\ 0 \\ f \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \text{unknown} \\ f \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ f \\ 1 \end{bmatrix} \Rightarrow \text{unknown} = f^2 \Rightarrow Af + B = f^2 \quad (3.30)$$

因此, 根据以上推导出的公式可以直接求解出, $A = n + f, B = -nf$, 进一步得到透视到正交的投影变换为,

$$M_{persp \rightarrow ortho} = \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n+f & -nf \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (3.31)$$

汇总以上结果，求出透视投影变换矩阵为，

$$M_{persp} = M_{ortho} M_{persp \rightarrow ortho} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{n-f} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -\frac{r+l}{2} \\ 0 & 1 & 0 & -\frac{t+b}{2} \\ 0 & 0 & 1 & -\frac{n+f}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n+f & -nf \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (3.32)$$

Chapter 4

Rasterization

4.1 Pixel and Screen

4.1.1 Screen

在执行 MVP (Model, View, Projection transformation) 之后，会得到一个标准的单位立方体： $[-1, 1]^3$ ，现在就需要将立方体画在屏幕上。

屏幕 (Screen) 是指包含一些列像素点的二维数组，屏幕分辨率指数组的大小 (长 $\times$ 宽)， $1080 \times 720p$ 简称为 $720p$ ， $1920 \times 1080p$ 简称为 $1080p$ ，更高有 $2k, 4k$ ，因此它是一种光栅化设备。

光栅 (Raster) 在德语中意指“screen”，光栅化 (Rasterize) 即将图形画在屏幕上。

像素 (Pixel) 即“Picture Element”的简称，是一个具有统一颜色的小正方块，由三原色 RGB (red, green, blue) 构成。

4.1.2 Screen Space

如下图所示，对应的每一个像素单元都定义了一个对应的像素位置，默认从 0 开始计数，因此对应的位置为像素单元左下角的坐标，例如蓝色像素单元的位置为 (2, 1)。

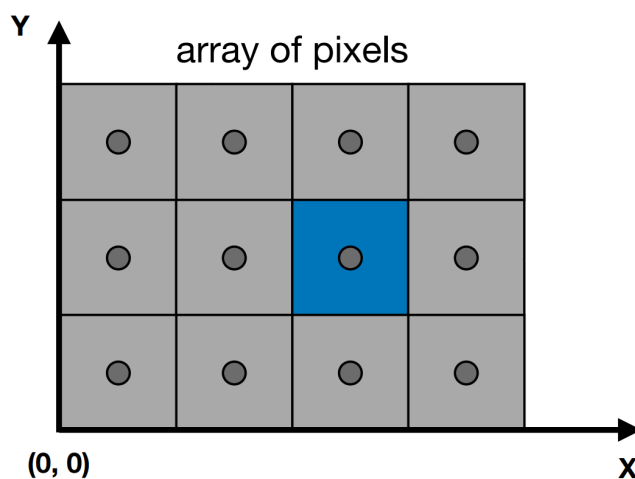


Figure 4.1: 像素空间坐标系

假设一个屏幕的分辨率为 $W \times H$ ，则对应的像素空间为： $[0, W - 1] \times [0, H - 1]$ 。但是每个像素的中心都在 $+0.5$ 偏移位置上，即像素 (x, y) 对应的像素中心位置为 $(x + 0.5, y + 0.5)$ 。对应屏幕在像素空间中覆盖的范围为， $[0, W] \times [0, H]$ 。

4.1.3 Viewport Transformation

通过以上章节定义了屏幕的像素空间，现在需要将经过 MVP 得到的立方体变换到屏幕空间上，即做视图变换。

暂时不考虑 z 轴，在 $x \times y$ 平面上，将 $[-1, 1]^2$ 转换到 $[0, W] \times [0, H]$ ，则视图变换矩阵为：

$$M_{viewport} = \begin{bmatrix} \frac{W}{2} & 0 & 0 & \frac{W}{2} \\ 0 & \frac{H}{2} & 0 & \frac{H}{2} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4.1)$$

4.2 Triangles Rasterization

将图形变换到定义的像素屏幕空间后，对应的图形还是空间坐标系中定义的矢量图形，不能称之为图像。现在就需要将这些图形离散化到对应的像素网格中，这个过程就叫做光栅化，意指将图形转换为图像，显示在屏幕上。

4.2.1 Fundamental Shape Primitives

三角形是最基本的形状原语，作为最基本的多边形，能够构成其他任意多边形，具有独特的性质：

1. 三角形内部一定是平面的；
2. 三角形内外定义是非常明确的，不会受到凹凸性的干扰，并且可以利用叉积进行判断；
3. 在三角形的顶点上进行插值可以得到属性渐变的过程（重心插值）；

4.2.2 Sampling

采样是指将一个连续函数进行离散化得到一系列函数值的过程。对应于图形学中，是采样某个屏幕像素空间中的一部分，然后对于这样采样点进行相应处理，例如判断是否是在图形内部，以此将其转换为对应的像素点。

定义二值函数 $inside(tri, x, y)$ 如下所示：

$$inside(t, x, y) = \begin{cases} 1, & \text{if Point}(x, y) \text{ is in triangle } t, \\ 0, & \text{otherwise.} \end{cases} \quad (4.2)$$

采样区域可以认为是整个屏幕空间，对应每一个像素点都进行 $inside(tri, x, y)$ 判断，因此可以归纳为：Rasterization = Sampling A 2D Indicator Function. 见下述代码，

```
for (int x = 0; x < xmax; ++x)
    for (int y = 0; y < ymax; ++y)
        image[x][y] = inside(tri, x + 0.5, y + 0.5);
```

回顾：叉积判定点在三角形内外

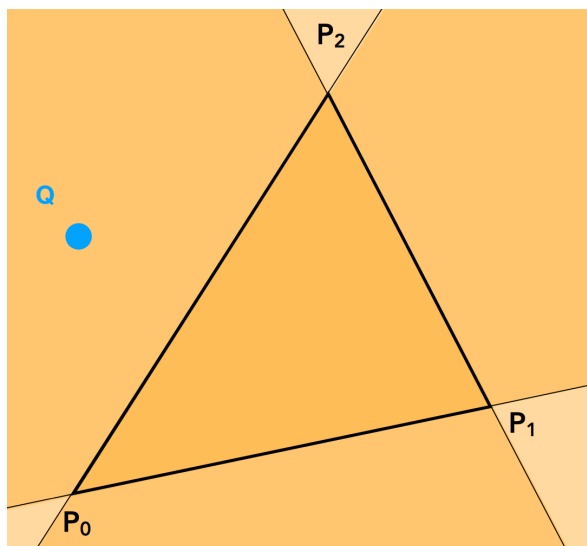


Figure 4.2: 三角形叉积判定内外

对于视图坐标系而言， z 轴指向纸面向外，判定点 Q 在 P_1P_2 的左右时，可以计算叉积 $\vec{P_1P_2} \times \vec{P_1Q}$ ，其中左右关系是相对于纸面面向相机的左右（默认相机是沿 $-z$ 方向）。

顺时针/逆时针标定三条边构成的向量后，以此判断 Q 点在三条边的左右关系，如果都在边的左侧/右侧，则 Q 点在三角形内部；否则在三角形外部。

4.2.3 Bounding Box Rasterization

根据上面光栅化过程，遍历整个屏幕空间判断像素点是否在三角形内部，将会额外遍历多个无关的像素点。一种解决的方式是，对于三角形添加一个边界框（如下图所示），分别取三个顶点两个维度的最大值和最小值，构成一个包围盒，随后在此包围盒中遍历像素点，将会降低一部分复杂度。

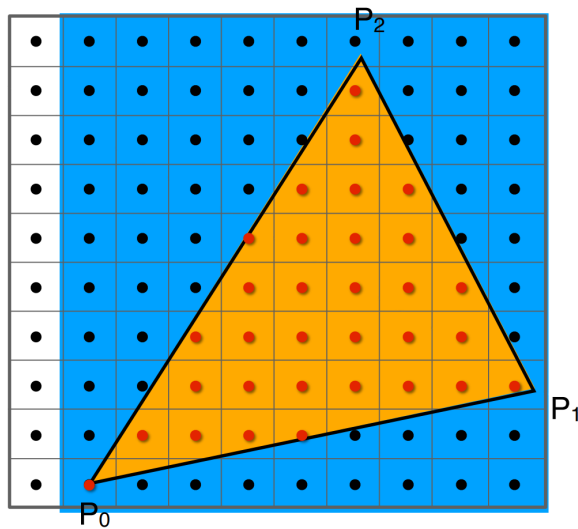


Figure 4.3: 三角形包围盒

4.3 Antialiasing and Z-Buffering

4.3.1 Antialiasing

当采样不恰当时，对应光栅化后的图形就会出现锯齿，锯齿的学名是走样（Aliasing）。这在计算机图形学中又被称为采样瑕疵 Sampling Artifacts (Errors/Mistakes/Inaccuracies)，具体体现有以下若干方面：

1. 锯齿 Jaggies，在像素空间中欠采样；
2. 摩尔纹 Moire，在图像中欠采样；
3. 齿轮效应 Wagon wheel effect，在时间上欠采样；

这些现象的出现，均是因为信号频率较高，而采样频率较低，出现欠采样现象。

Frequency Domain

信号处理背景准备/回顾：数字信号处理笔记。

一种有效的反走样方法是，对信号做模糊（滤波），再做采样。由数字信号处理知识可以分析得到，对信号做模糊处理，实际上是施加了一个低通滤波器，滤除了原始信号的高频部分，只保留低频部分，这样信号的截止频率即最高频率就会降低，那么在采样频率保持不变的条件下，更容易满足采样定理，更不会发生失真现象即走样现象。

在博客文章 [Signal Filters Design Based on Digital Signal Processing](#) 中，我也深入地分析了四种数字滤波器（低通，带通，带阻，高通）的滤波特性以及频谱分析，探究了其采样定理的满足条件。

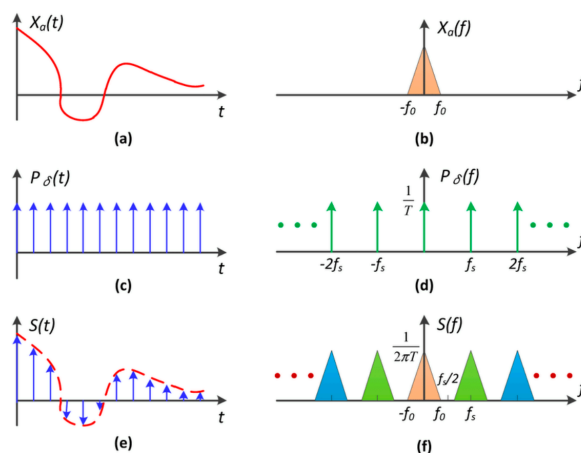


Figure 4.4: 利用冲激函数进行信号采样

如上图所示，对于一个信号而言，可以将其与冲激函数在时域相乘表示信号的采样过程，根据卷积定理：时域相乘，频域相卷积。在频域中，相当于两个信号的频谱相卷积，可以分析得到，信号采样即是在频域上以采样频率为间隔重复其频谱。

Mixed Frequency

采样定理表述为， $f \leq \frac{f_s}{2}$ ，即信号的截止频率需要小于或等于采样频率的一半。

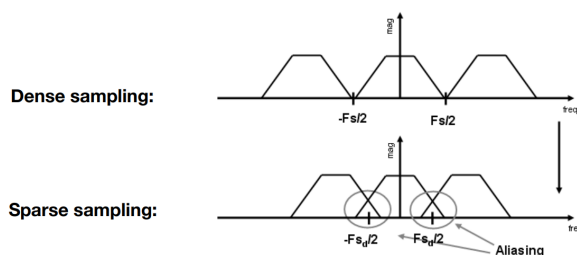


Figure 4.5: Dense VS Sparse Sampling

当满足采样定理（Dense Sampling）时，对应的采样频谱不会发生混叠现象，而相反，当不满足采样定理（Sparse Sampling）时，对应的采样频谱发生混叠现象。

Reduce Aliasing

从信号处理的角度，解决采样信号频谱混叠的有效方法无非是有两个方面，一个方面从采样方法分析，提高采样技术，即增大采样频率；另一个方面从信号本身分析，源头上降低信号截止频率，即进行滤波。

在图形学中的应用也是如此，但是对于一个显示设备而言，采样率就是屏幕的分辨率，通常设备无法进一步提高分辨率，因此往往通过第二种方法进行反走样。

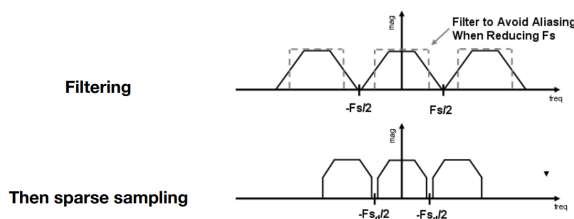


Figure 4.6: 低通滤波进行采样

根据上图可以看到，通过在采样之前对信号进行低通滤波，可以过滤掉一些高频部分，使得信号的截止频率进一步降低，这将有利于满足采样定理，使得信号不易失真。

Antialiasing By Supersampling (MSAA)

像素滤波器（1 pixel-width box filter）在实际中应用滤波，一般是通过设置一个像素点大小的滤波器，然后采样该像素格子中的像素值作为该像素的像素值。

超采样是指对于一个像素格子采样多个位置，然后计算平均值。

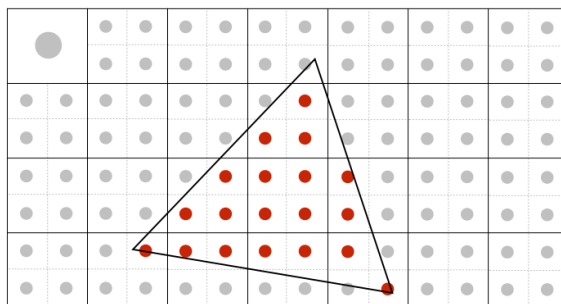


Figure 4.7: 超采样位置分布

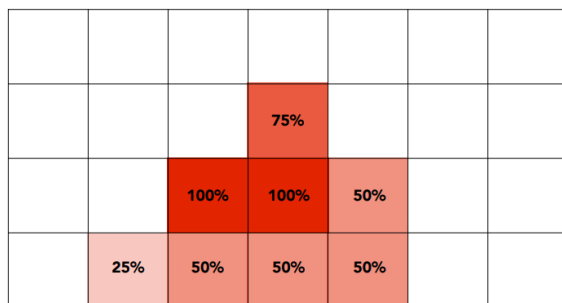


Figure 4.8: 超采样像素百分比

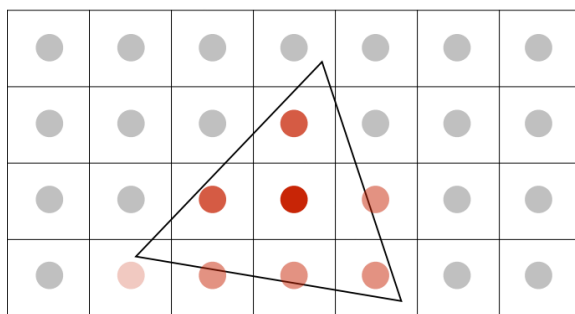


Figure 4.9: 超采样像素平均值

根据上图可以看到，对于超采样方法而言，通过对一个像素格子进行采样多个位置，然后通过计算像素格子中平均值作为该像素格子的像素值，这将对图像边缘进行模糊化，有利于抗拒反走样。

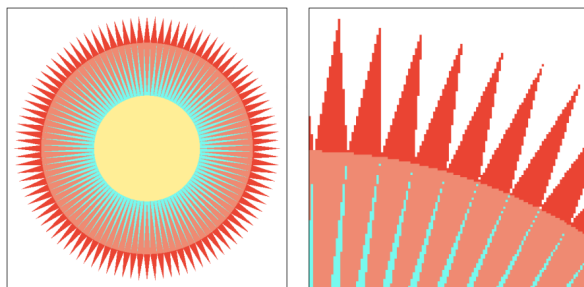
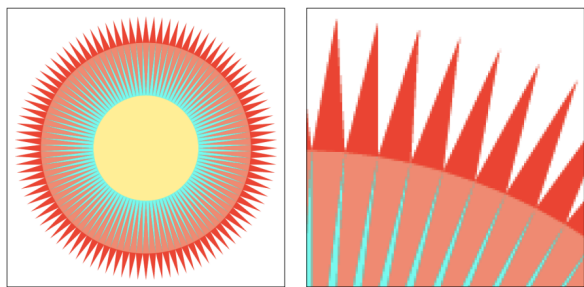


Figure 4.10: Point Sampling

Figure 4.11: 4×4 Supersampling

4.3.2 Z-Buffering

深度缓冲技术 (Z-Buffering) 是为了解决在屏幕上显示存在相互遮挡物体先后顺序的问题, 是一种非常高效的方法。它的引入是通过对单个的像素进行“排序”, 实际上是指记录每个像素上最小的 z 值, 即距离相机最近的深度, 对应位置的物体作为最前面的物体显示在屏幕上。

这种方法需要引入额外的存储空间记录每个像素上的最小的深度, 因此对于一张光栅化的图像而言, 具有 Frame Buffer 用于记录每个像素的像素值, 一个深度图 Depth Buffer 用于记录每个像素对应的最小深度。当光栅化另一个物体时, 就可以利用这两个 Buffer 进行更新, 就可以处理物体遮挡问题。

这里的 z 值不同于相机坐标系中沿向 $-z$ 方向的坐标, 而是一个正数值, 越小对应的深度越接近屏幕, 越大对应的深度越远离屏幕。

因此光栅化每个三角形时, 对应的 Z-Buffer 算法可以简单地表述为:

```
for (each triangle T)
  for (each sample (x,y,z) in T)
    if (z < zbuffer[x,y]){ // closest sample so far
      framebuffer[x,y]=rgb; // update color
      zbuffer[x,y]=z; // update depth
    }
  else continue;
```

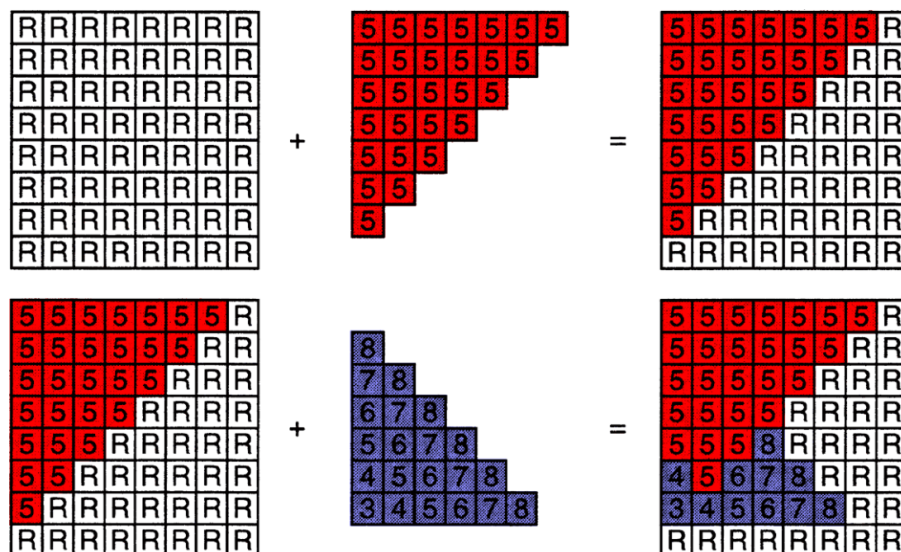


Figure 4.12: Z-Buffer 处理三角形光栅化遮挡问题

如图 4.12 可以看到, Z-Buffer 利用深度缓存处理两个三角形的遮挡问题, 会不断地更新屏幕上覆盖到的所有像素的最小深度。

4.4 Shading

着色 (Shading) 问题是指对物体增添明暗或颜色效果, 在图形学中是对不同的物体施加不同的材质, 实际上是对不同的光照进行处理。更加形象地描述, shading 是在解决 how light interacts with matter 的问题。

*Shading is the process in rendering that involves computing the color of objects in a 3D scene. Rendering aims to reproduce the **shape, visibility, and appearance** of objects as seen from a given viewpoint.*

*In computer graphics, a shading function is defined as a function that yields **the intensity value of each point on an object's body**, based on the characteristics of the light source, the object, and the position of the observer. - Phong, 1975.*

在真实场景中，光照总是会在物体的表面上发生反射，但是由于绝对光滑的表面是不存在的，因此更多情况是光线在物体表面发生了漫反射。根据大学物理可以得知，光的漫反射是均匀地向各个方向进行反射的。

由于位置和角度的特异性，物体不同的着色点能够接收到不同强度和数量的漫反射光，因此会形成明暗和颜色的变化。

Rendering VS Shading:

- 在图形学中，Rendering 分为真实感渲染 (Photorealistic Rendering) 和非真实感渲染 (Non-Photorealistic Rendering) 两种，但一般关注于真实感渲染。
- The goal of **Photorealistic Rendering** is to produce computer-generated images that look real or resemble photographs.
- **Shading**, which defines the appearance of objects during the rendering process, **plays a critical role in photorealistic rendering.**
- Shading 是 Rendering 中的重要一部分。

真实感渲染就是在计算机中模拟现实世界中物体的外观，这主要由光照 (**lighting**) 和物体的属性 (**properties**) 影响。因此，渲染就是计算机就是在准确地模拟这些因素以及它们之间的交互作用。

其中，物体的属性可以分为两种：物体表面的几何属性（例如物体的朝向）以及影响光和物体如何交互的内在属性（例如物体的颜色）。

4.4.1 Interaction between Lighting and Objects

光与物体的直接作用方式主要由物体的表面类型所决定，主要有镜面反射、光泽作用以及漫反射三种类型。

- **Mirror:** 镜面反射是指物体表面是一个理想的镜面，满足镜面反射定律。
- **Shininess or Glossiness:** 物体是光泽面，可以认为是破碎的镜子，由许许多多微小镜子组成，朝向随机的方向，反射光也朝向各个方向射出。当物体表面粗糙时，出射光会稍微偏离镜面反射的方向，使得反射图像变得模糊。而且物体表面越粗糙，反射越模糊。因此，物体的光泽度是视角依赖型的。
- **Diffuse or Lambertian:** 漫反射的特性往往与表面粗糙度无关，而是具有复杂的内部结构，会导致入射光被困住，并在材料内部多次反弹，使得出射方向与入射方向无关。在 CG 中，通常认为漫反射的出射光是随机的，等概率地向四面八方随机射出。因此，漫反射光强不会因为视角的变化而改变。

物体呈现的颜色，是由于物体自身的属性决定的，吸收白光中特点颜色（频谱）的光后，剩余反射出来的颜色光的混合就是我们看到的物体的颜色。

4.4.2 Blinn-Phong Reflectance Model

通过图 4.13 所示，可以直观理解几个光线部分：高光（Specular highlights），漫反射（Diffuse reflection），环境光（Ambient lighting）。



Figure 4.13: Shading 的直观理解

朗伯（漫反射）着色模型针对漫反射光的特性，提出了下面的公式以处理经过漫反射后的光强。

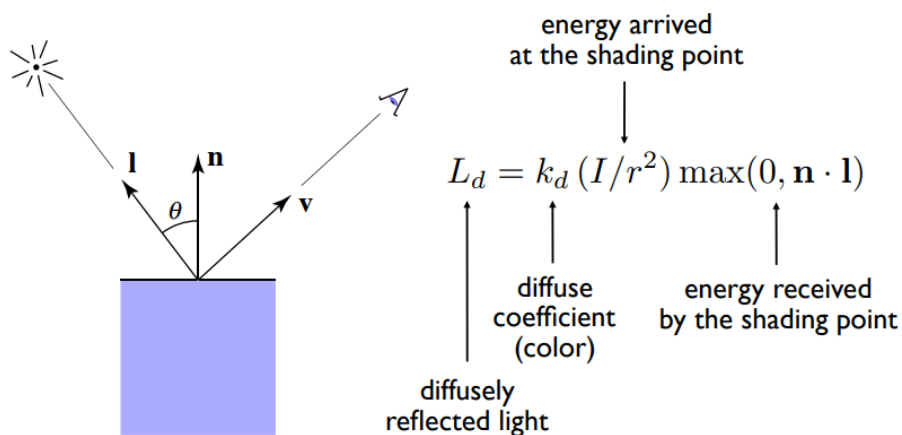


Figure 4.14: Lambertian (Diffuse) Shading

其中， v 表示视角方向，即相机接收到反射光的方向， n 表示局部表面（切平面）的法线， l 表示入射光的方向，Shading Point 表示一个非常局部的着色点，并且只考虑光照与着色点的关系，因此不会产生阴影。 k_d 表示与物体表面材质有关的漫反射系数，用于建模不同材质对光照的吸收程度。

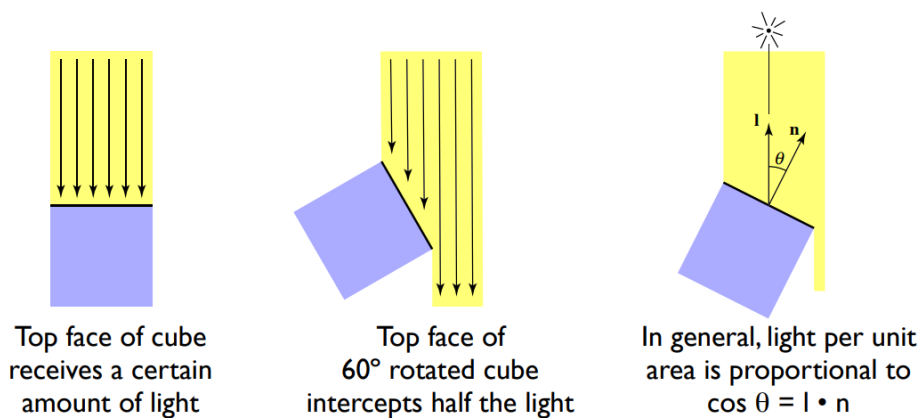


Figure 4.15: Lambert's Cosine Law

如图 4.15 所示，朗伯余弦定律描述了理想漫反射表面的光辐射强度分布特性，单位面积上的光强与平面和光照夹角的余弦成正比。

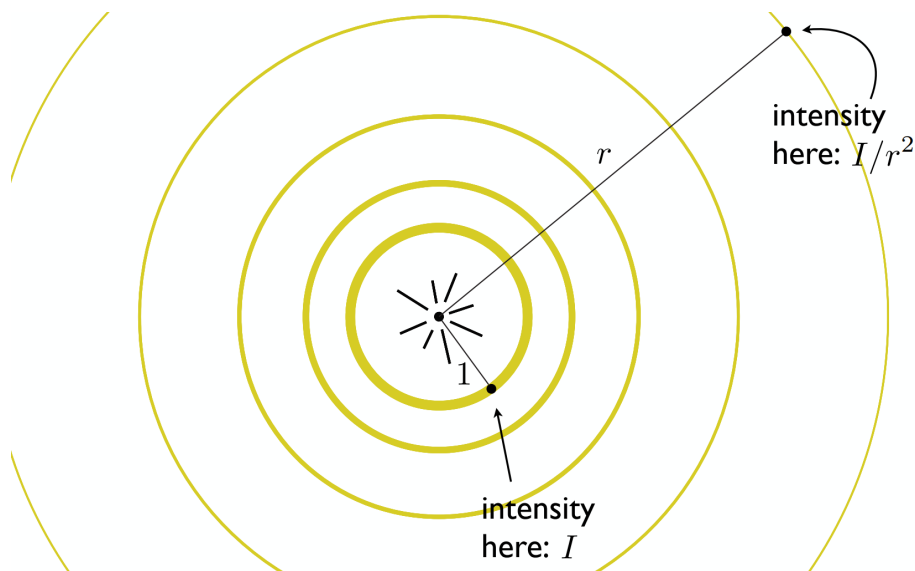


Figure 4.16: 光强衰减规律

光强衰减是指相对于距离光源单位距离时单位表面积上的光强 I ，在传播距离为 r 后得到的衰减光强 I' 与 r^2 成反比，如图 4.16 所示。